📏 Linear Velocity Calibration

1. Course Introduction

Use a physical reference object of known length such as a tape measure to verify the actual translational distance measurement accuracy of the A1 Pro chassis wheel odometry (calculated from chassis MCU encoder and front wheel steering data), and precisely calibrate the linear velocity scale correction factor.

🎯 Course Objectives

1. Understand the principles and calculation formulas for linear velocity calibration based on Ackerman chassis wheel odometry
2. Master the calibration operation process using the `calibrate_linear_A1P` node combined with a physical tape measure
3. Master the persistence method for writing the optimal correction factor to the low-level driver parameter configuration file `a1_driver.yaml`

> **Important**
>
> The vehicle has been calibrated at the factory before shipment, so **there is no need to recalibrate** for first-time use. You only need to refer to this tutorial for recalibration when long-term operation causes wear, when different tires are replaced, or when a fixed proportional cumulative error is observed in the vehicle traveling a specific distance.

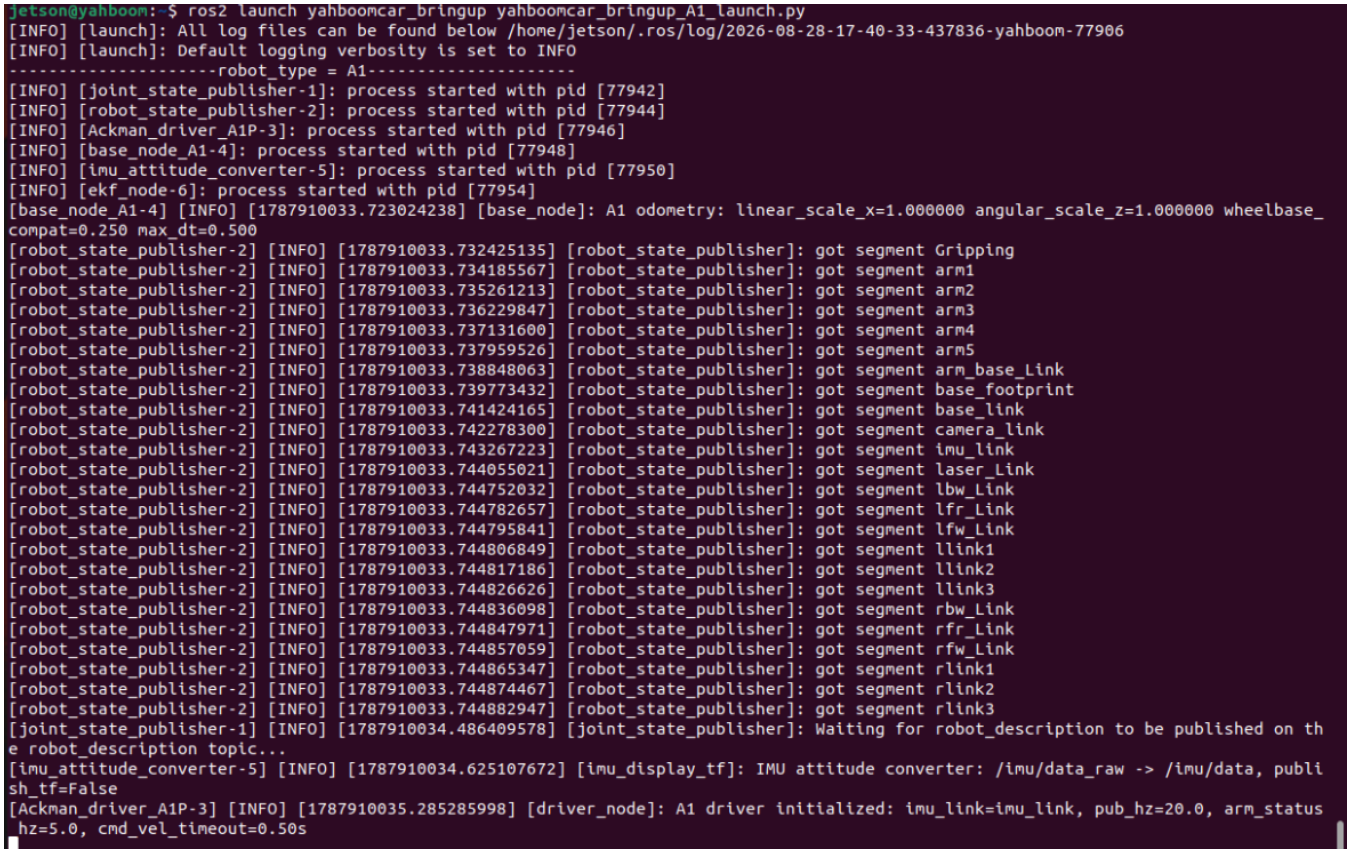
---

2. Preparation Work

2.1 Start the Chassis Integrated Driver

Open a terminal and run the following command to start all A1 Pro chassis drivers and the EKF fusion localization node:

```
ros2 launch yahboomcar_bringup yahboomcar_bringup_A1_launch.py
```



2.2 Prepare the Measurement Environment and Tools

- Prepare a tape measure accurate to the millimeter (at least 2 meters long);
- Select a flat, unobstructed open area, use tape to mark a starting line on the ground, and align the front wheel (or rear wheel axle vertical line) of the vehicle with the starting line.

---

3. Calibration Process

3.1 Start the Linear Velocity Calibration Program

Run the calibration test node in a new terminal:

```
ros2 run yahboomcar_bringup calibrate_linear_A1P
```

```
jetson@yahboom:~$ ros2 run yahboomcar_bringup calibrate_linear_A1
[INFO] [1787910094.123895565] [calibrate_linear]: Waiting for TF transforms to be ready...
[INFO] [1787910094.124864615] [calibrate_linear]: ================================================
[INFO] [1787910094.125432438] [calibrate_linear]: Linear calibration node started
[INFO] [1787910094.126162666] [calibrate_linear]: Start testing with command:
[INFO] [1787910094.126832700] [calibrate_linear]:   ros2 param set /calibrate_linear start_test True
[INFO] [1787910094.127374219] [calibrate_linear]: ================================================
[INFO] [1787910094.146217484] [calibrate_linear]: TF transform ready
[INFO] [1787910105.328415731] [calibrate_linear]: Starting test #1
[INFO] [1787910105.329884763] [calibrate_linear]:   Target distance: 1.0m
[INFO] [1787910105.331221951] [calibrate_linear]:   Scale correction: 1.0
[INFO] [1787910105.332487810] [calibrate_linear]:   Start position: (0.013, 0.006)
[INFO] [1787910109.285828092] [calibrate_linear]: Test #1 complete
[INFO] [1787910109.286534735] [calibrate_linear]:   Expected distance: 1.000m
[INFO] [1787910109.287148768] [calibrate_linear]:   Actual distance (uncorrected): 0.994m
[INFO] [1787910109.287725776] [calibrate_linear]:   Suggested scale correction: 1.0057
[INFO] [1787910109.289231161] [calibrate_linear]: ================================================
[INFO] [1787910109.289826665] [calibrate_linear]: Calibration complete!
[INFO] [1787910109.290382392] [calibrate_linear]: ------------------------------------------------
[INFO] [1787910109.291183438] [calibrate_linear]: Test results summary:
[INFO] [1787910109.292009284] [calibrate_linear]:   Test 1: Expected 1.000m, Actual 0.994m, Scale 1.0057
[INFO] [1787910109.292781241] [calibrate_linear]: ------------------------------------------------
[INFO] [1787910109.293635632] [calibrate_linear]: Suggested scale correction: 1.0057
[INFO] [1787910109.294468263] [calibrate_linear]: ------------------------------------------------
[INFO] [1787910109.295346303] [calibrate_linear]: Please add the following parameter to your configuration file:
[INFO] [1787910109.296214614] [calibrate_linear]:   odom_linear_scale_correction: 1.0057
[INFO] [1787910109.297077678] [calibrate_linear]: ================================================
```

The calibration node will listen to the real-time pose transformation from `odom -> base_footprint` in the TF tree, and obtain the relative displacement reported by the odometry.

3.2 Trigger the Straight-Line Driving Test

In another terminal, trigger the test through a ROS 2 dynamic parameter:

```
ros2 param set /calibrate_linear start_test True
```

```
jetson@yahboom:~$ ros2 param set /calibrate_linear start_test True
Set parameter successful
jetson@yahboom:~$
```

The vehicle will drive straight forward at a preset speed (default 0.2~0.3 m/s). When the odometry reports the displacement reaching the preset target distance (default `test_distance: 1.0` meters), it will automatically brake to a stop and output the test results in the terminal.

3.3 Measure the Actual Physical Distance on the Ground

After the vehicle comes to a complete stop, use the tape measure to measure the **actual physical movement distance (Actual Distance)** of the vehicle reference point (such as the wheel axle center) from the starting line to the current stopping position.

3.4 Calculate and Dynamically Adjust the Correction Factor

The calibration program calculates displacement based on odometry feedback. In the program and low-level driver, the odometry displacement is **multiplied by the correction factor** (that is: **determined displacement = odometry displacement * correction factor**):

- **Writing a factor greater than 1.0**: The odometry reading is amplified, displacement accumulates faster, the vehicle will **reach the target value earlier and brake earlier** (the actual distance traveled on the ground becomes **shorter**);
- **Writing a factor less than 1.0**: The odometry reading is reduced, displacement accumulates slower, the vehicle needs to travel longer to reach the target value, the vehicle will **brake later** (the actual distance traveled on the ground becomes **longer**).

Therefore, based on the measured physical tape measure value on the ground and the expected target distance, the new correction factor calculation formula is:

```
New Correction Factor = Current Factor * (Actual Physical Measured Distance / Target Travel Distance)
```

Parameter Adjustment Example and Calculation:

- **Scenario 1: Overshot (Actual Distance > Target Distance)**
  Set target travel to 1.000 meters, current factor is 1.0, vehicle actually measured and moved 1.050 meters (overshot the line by 5cm):

  ```
  New Factor = 1.0 * (1.050 / 1.000) = 1.0500 (Increase the factor)
  ```

  After increasing the factor from 1.0 to 1.0500, when the vehicle actually travels 1.0 meters, the odometry will prematurely report the full `1.0 * 1.05 = 1.05` meters, thus braking earlier and stopping precisely at the 1.0 meter line.

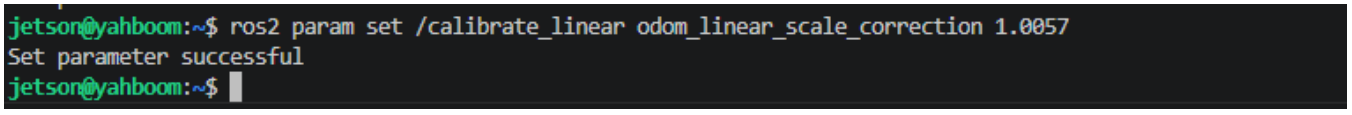
- **Scenario 2: Under-shot (Actual Distance < Target Distance)**
  Set target travel to 1.000 meters, current factor is 1.0, vehicle only actually moved 0.950 meters (stopped early, still 5cm short of the line):

  ```
  New Factor = 1.0 * (0.950 / 1.000) = 0.9500 (Decrease the factor)
  ```

  After decreasing the factor from 1.0 to 0.9500, the odometry reading slows down, the vehicle will continue forward to make up the extra 5cm travel, precisely reaching 1.0 meters.

Dynamically set the new factor via command line (example of increasing to 1.0500):

```
ros2 param set /calibrate_linear odom_linear_scale_correction 1.0500
```



> **Tip**
>
> Core rule:
>
> - If the actual physical measured distance on the ground is **greater than** the target distance (overshot) **Increase** the correction factor;
> - If the actual physical measured distance on the ground is **less than** the target distance (under-shot) **Decrease** the correction factor.

3.5 Multiple Verifications and Convergence

Move the vehicle back to the starting line, run `ros2 param set /calibrate_linear start_test True` again, and observe whether the vehicle stops precisely at the 1.000 meter mark after applying the new correction factor. Repeat until the actual physical error is within 5 millimeters.

---

4. Persist the Correction Factor to Driver Configuration

After calibration is complete, to ensure the low-level driver automatically applies the parameters on every boot or launch, write the optimal factor obtained from calibration to the parameter configuration file of the `driver_node`:

Parameter file path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/a1_driver.yaml
```

Open `a1_driver.yaml`, find the `odom_linear_scale_x` configuration item and modify it to the final calibrated value (for example 1.0500):

```yaml
driver_node:
  ros__parameters:
    # ...
    # Chassis linear velocity odometry scale correction factor (new value after calibration)
    odom_linear_scale_x: 1.0500
    odom_angular_scale_z: 1.0
    # ...
```

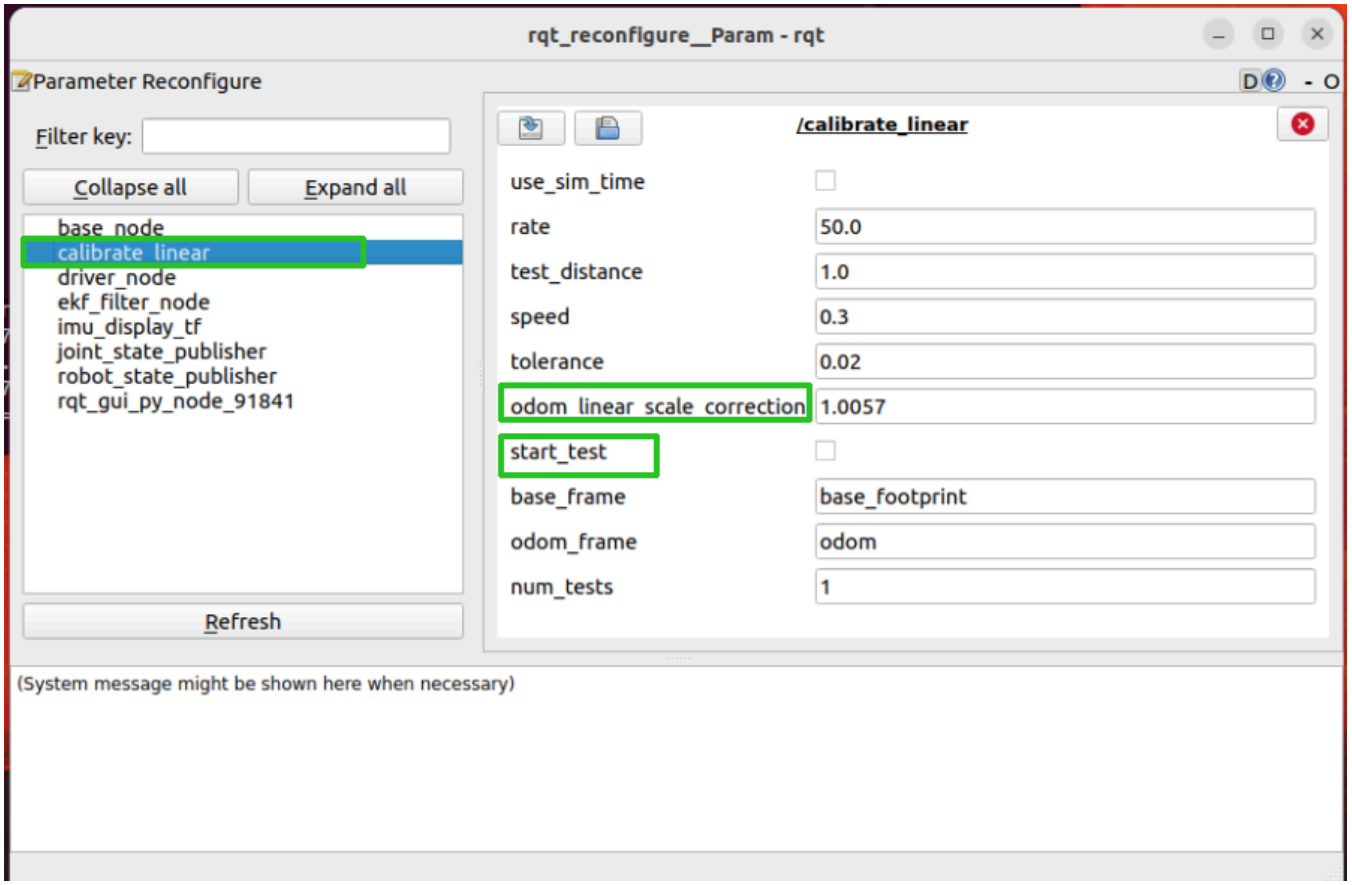
> **Note**
>
> After modifying `a1_driver.yaml`, it is recommended to recompile the package to synchronize the configuration file in the installation path:
>
> ```
> cd ~/yahboomcar_ros2_ws/yahboomcar_ws
> colcon build --packages-select yahboomcar_bringup
> source install/setup.bash
> ```

---

5. Using rqt_reconfigure for Graphical Parameter Tuning

In addition to the command line, you can also use the `rqt_reconfigure` graphical interface provided by ROS 2 for visual real-time parameter tuning and test triggering:

```
ros2 run rqt_reconfigure rqt_reconfigure
```



> **Important**
>
> **What to do when no nodes are displayed in the interface**:
> If after opening `rqt_reconfigure`, the `/calibrate_linear` node is **not shown** in the list on the left side, this is due to the timing sequence between node startup and tool loading not auto-refreshing. Simply click the **Refresh button (Refresh)** in the upper left corner of the rqt interface to rescan and display `/calibrate_linear`.

5.1 Calibration Node Parameter Description

The `/calibrate_linear` node panel includes the following key parameters:

Parameter Name	Default Value	Physical Meaning and Description
`start_test`	`False`	Start test trigger switch (check to start test, automatically unchecks when travel is complete)
`odom_linear_scale_correction`	`1.0`	Linear velocity scale correction factor for real-time testing within the calibration node (>1.0 brakes earlier and travels shorter, <1.0 delays braking and travels farther)
`test_distance`	`1.0`	Target distance for test travel (meters)
`speed`	`0.2`	Test travel linear velocity (m/s)

Parameter Name	Default Value	Physical Meaning and Description
`tolerance`	`0.02`	Position error dead zone tolerance for target distance arrival determination (meters)
`num_tests`	`1`	Number of consecutive test rounds
`rate`	`50.0`	TF query and control main loop frequency (Hz)
`base_frame`	`base_footprint`	Robot chassis projection base coordinate frame name
`odom_frame`	`odom`	Odometry global reference coordinate frame name

6. Core Source Code and Algorithm Principle Analysis

Source code path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/yahboomcar_bringup/calibrate_linear_A1P.py
```

6.1 Calibration Closed-Loop Control Core Logic

```
def on_timer(self):
    if self.is_testing:
        x, y = self.get_position()
        if x is None:
            return

        # 1. Calculate cumulative displacement from starting point
        distance = sqrt((x - self.x_start)**2 + (y - self.y_start)**2)
        # 2. Multiply odometry raw displacement by correction factor
        corrected_distance = distance * odom_scale
        error = test_distance - corrected_distance

        # 3. Closed-loop control for straight-line movement, keep publishing linear velocity until target distance is reached
        if abs(error) > tolerance:
            move_cmd = Twist()
            move_cmd.linear.x = copysign(speed, error)
            self.cmd_vel_pub.publish(move_cmd)
        else:
            # 4. Target distance reached, publish brake stop
            self.cmd_vel_pub.publish(Twist())
            self.is_testing = False
```

6.2 Source Code and Driver Activation Mechanism Analysis

- **Why does the vehicle stop early when the factor is greater than 1.0**:
  The condition for the calibration program to determine stopping is `corrected_distance >= test_distance`, i.e., `distance * odom_scale >= test_distance`.
  Therefore, when the vehicle stops, the odometry displacement is `distance = test_distance / odom_scale`.
  When `odom_scale > 1.0`, the denominator becomes larger, and the odometry only needs to accumulate a smaller displacement to determine reaching the target, thus **braking earlier and stopping**.
- **Low-level driver activation mechanism**:
  In the chassis low-level driver `Ackman_driver_A1P.py`:

  ```
  vx = float(vx) * self.odom_linear_scale_x
  ```

  The driver node directly multiplies the speed reported by the MCU by `odom_linear_scale_x`. When a factor greater than 1.0 is written, the odometry displacement and speed reported to ROS 2 are proportionally amplified. After the optimal factor obtained from calibration is written to `a1_driver.yaml`, the odometry accuracy of the entire ROS 2 navigation and mapping localization stack will be perfectly aligned with the physical real world.